ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Benchmarking the Recommendation Spectrum

ONE LOVE. ONE FUTURE.

Introduction

Introduction: Team member

Nguyen Duy Tung - 202416762

Le Tuan Hung - 202416800

Phung Thanh Quang - 202416822

Le Van Tuan Anh - 202416657

Tran Hong Ha - 202416687

Introduction

Machine learning Approach:

- Content-based Filtering
- Memory - based Filtering
- Hybrid Method
- Matrix Factorization

Deep learning Approach

- Neural Collaborative Filtering
- Light GCN
- Two - Towers
- SAS Rec

Dataset

MovieLens-1M

Consist of:

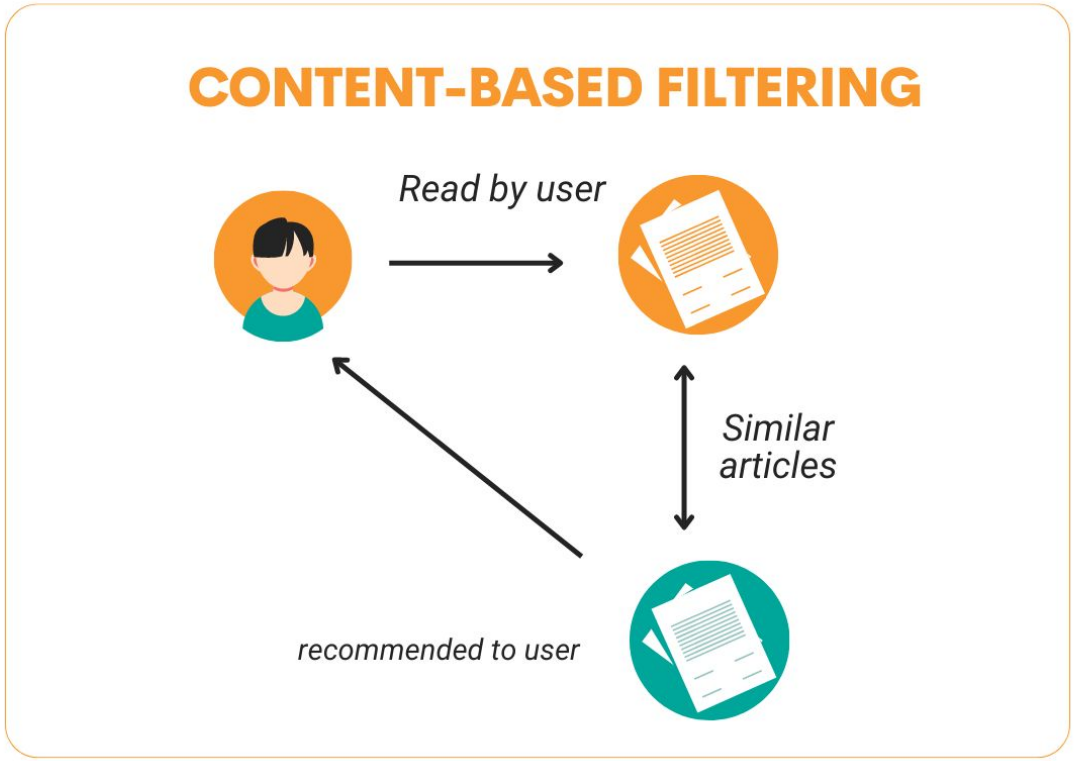
- 1 million explicit ratings
- ~ 4,000 movies and 6,000 users
- User age, gender, occupation, zip code
- Movie title, release year, genres

Content - based Filtering

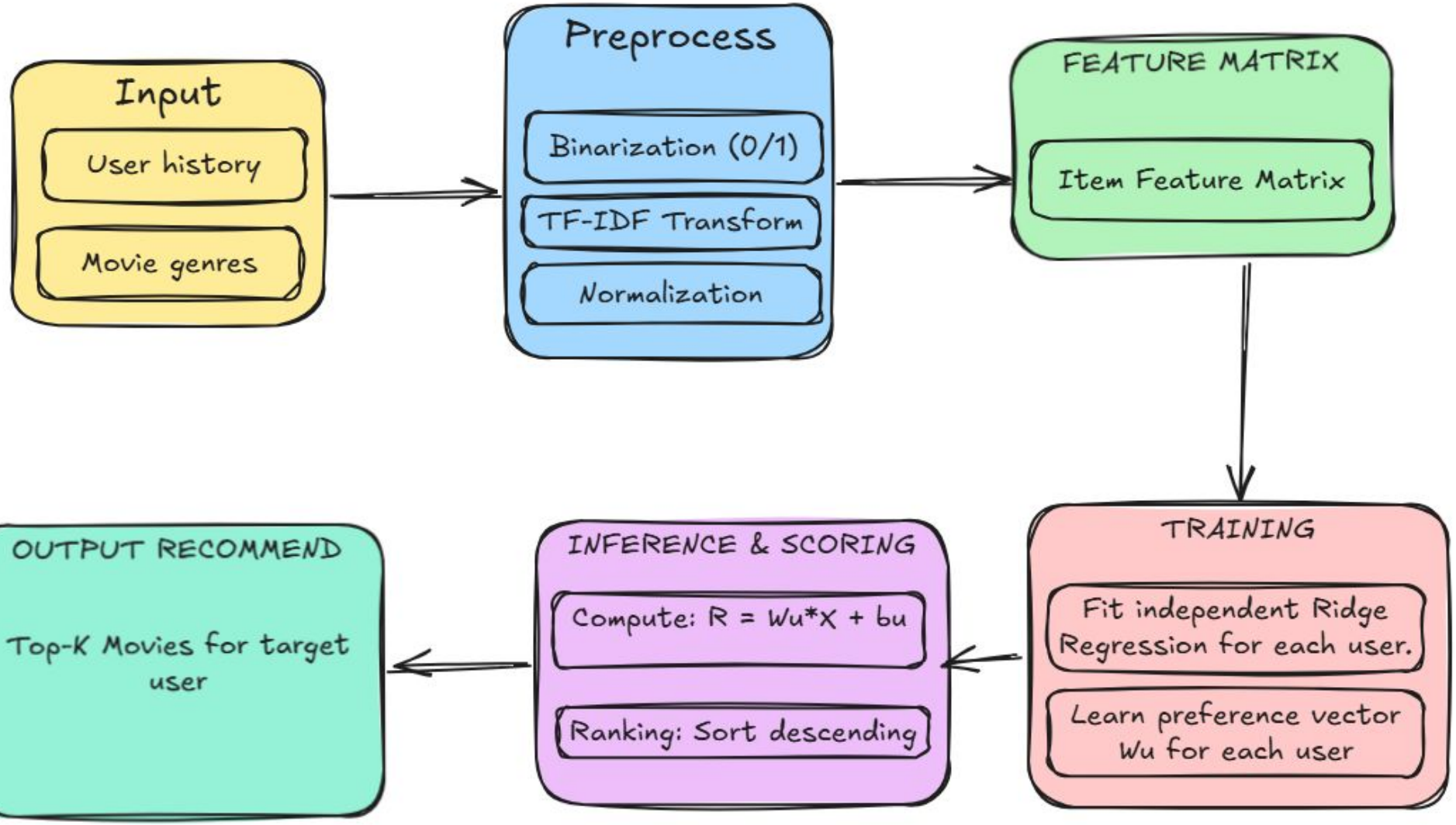
Overview

Idea: Recommend movies similar to movies the user liked before.

Input: Movie genres (Genre vectors)



Pipeline & Architecture



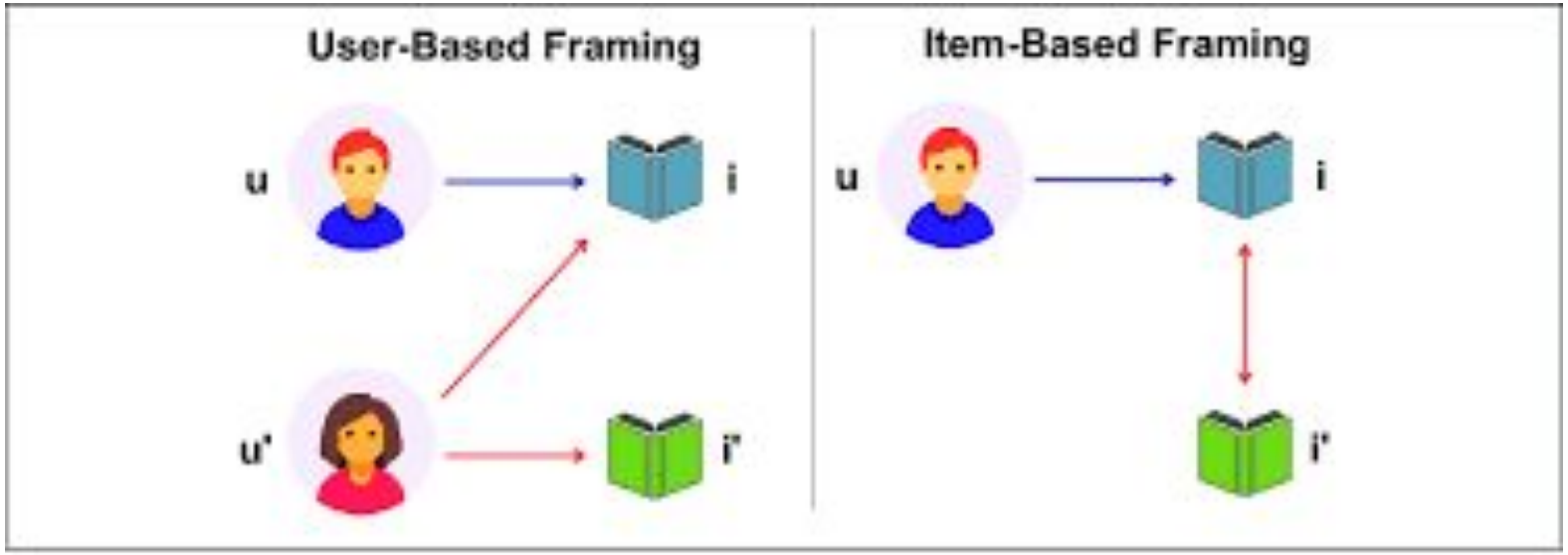
Memory - based Filtering (User - based/Item - based)

Overview

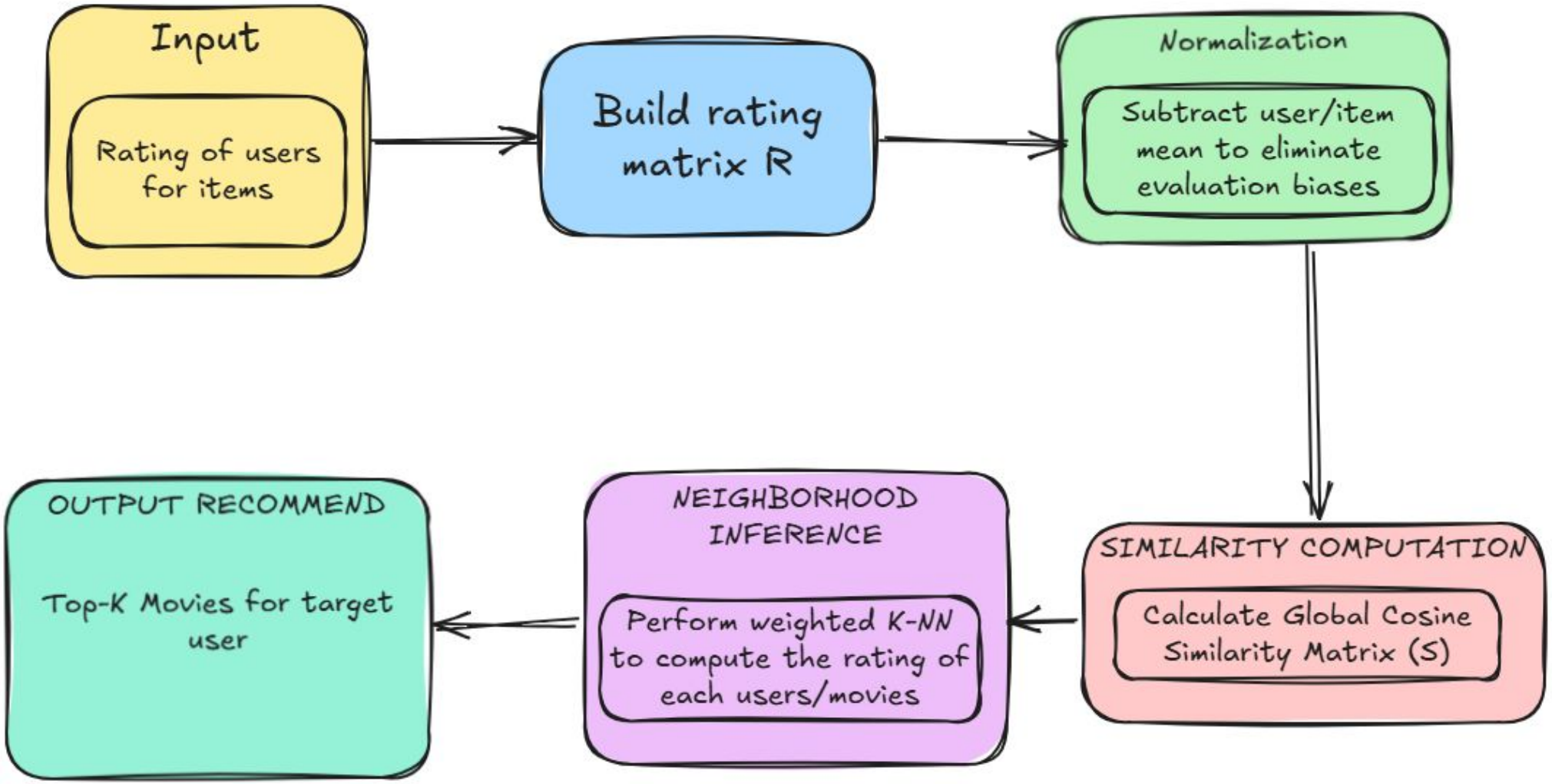
Idea: Users with similar behavior in the past tend to give similar ratings in the future and same with items

Input: Rating from each users for each movies (Rating matrix)

Intuition: Essentially an extension of weighted K-Nearest Neighbors (Weighted K-NN)



Pipeline


Input
Rating of users for items
Build rating matrix R
Normalization
Subtract user/item mean to eliminate evaluation biases
SIMILARITY COMPUTATION
Calculate Global Cosine Similarity Matrix (S)
NEIGHBORHOOD INFERENCE
Perform weighted K-NN to compute the rating of each users/movies
OUTPUT RECOMMEND
Top-K Movies for target user

Example

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	5	5	2	0	1	?	?
i_1	4	?	?	0	?	2	?
i_2	?	4	1	?	?	1	1
i_3	2	2	3	4	4	?	4
i_4	2	0	4	?	?	?	5
	↓	↓	↓	↓	↓	↓	↓
$\bar{u}_j$	3.25	2.75	2.5	1.33	2.5	1.5	3.33

a) Original utility matrix $\mathbf{Y}$ and mean user ratings.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	1.75	2.25	-0.5	-1.33	-1.5	0	0
i_1	0.75	0	0	-1.33	0	0.5	0
i_2	0	1.25	-1.5	0	0	-0.5	-2.33
i_3	-1.25	-0.75	0.5	2.67	1.5	0	0.67
i_4	-1.25	-2.75	1.5	0	0	0	1.67

b) Normalized utility matrix $\bar{\mathbf{Y}}$.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
u_0	1	0.83	-0.58	-0.79	-0.82	0.2	-0.38
u_1	0.83	1	-0.87	-0.40	-0.55	-0.23	-0.71
u_2	-0.58	-0.87	1	0.27	0.32	0.47	0.96
u_3	-0.79	-0.40	0.27	1	0.87	-0.29	0.18
u_4	-0.82	-0.55	0.32	0.87	1	0	0.16
u_5	0.2	-0.23	0.47	-0.29	0	1	0.56
u_6	-0.38	-0.71	0.96	0.18	0.16	0.56	1

c) User similarity matrix $\mathbf{S}$.

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	1.75	2.25	-0.5	-1.33	-1.5	0.18	-0.63
i_1	0.75	0.48	-0.17	-1.33	-1.33	0.5	0.05
i_2	0.91	1.25	-1.5	-1.84	-1.78	-0.5	-2.33
i_3	-1.25	-0.75	0.5	2.67	1.5	0.59	0.67
i_4	-1.25	-2.75	1.5	1.57	1.56	1.59	1.67

d) $\hat{\mathbf{Y}}$

Predict normalized rating of u_1 on i_1 with $k = 2$

Users who rated i_1 : $\{u_0, u_3, u_5\}$

Corresponding similarities: {0.83, -0.40, -0.23}

$\Rightarrow$ most similar users: $\mathcal{N}(u_1, i_1) = \{u_0, u_5\}$

with **normalized ratings {0.75, 0.5}**

$$\Rightarrow \hat{y}_{i_1,u_1} = \frac{0.83*0.75+(-0.23)*0.5}{0.83+|-0.23|} \approx 0.48$$

e) Example

	u_0	u_1	u_2	u_3	u_4	u_5	u_6
i_0	5	5	2	0	1	1.68	2.70
i_1	4	3.23	2.33	0	1.67	2	3.38
i_2	4.15	4	1	-0.5	0.71	1	1
i_3	2	2	3	4	4	2.10	4
i_4	2	0	4	2.9	4.06	3.10	5

f) Full $\mathbf{Y}$

Hybrid (Memory-based and Content-based Combined)

Idea: Weighted Hybrid

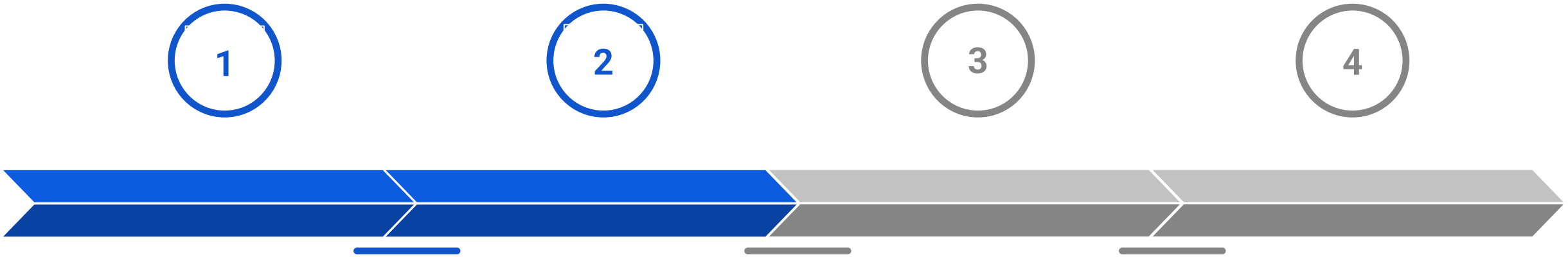
- Problems: Each standalone Memory-based and Content-based models have their own strengths and weaknesses
- Instead of trusting one model completely, we take a weighted average of all three prediction signals

  => Weighted Hybrid

$$\hat{R}_{hybrid} = \alpha \cdot \hat{R}_{item} + \beta \cdot \hat{R}_{user} + (1 - \alpha - \beta) \cdot \hat{R}_{content}$$

$$\text{where } \alpha \geq 0, \quad \beta \geq 0, \quad \text{and } \alpha + \beta \leq 1$$

Pipeline



Data Processing

Split into Train, Validation, and Test Set

Component Models Training

Train Memory and Content based models to get the predicted results

Hyperparameter Tuning

Weighted hybrid + Grid search for best weights combination

Inference

Upon request, give users the weights combination for them to blend the result

Matrix Factorization

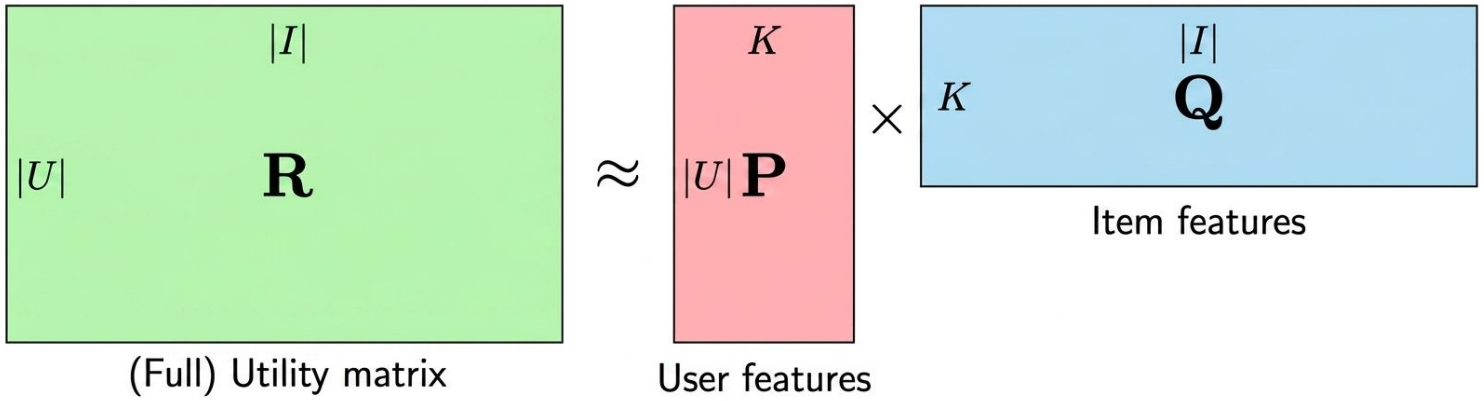
Overview

Idea: The existence of latent features describing the user-item interaction

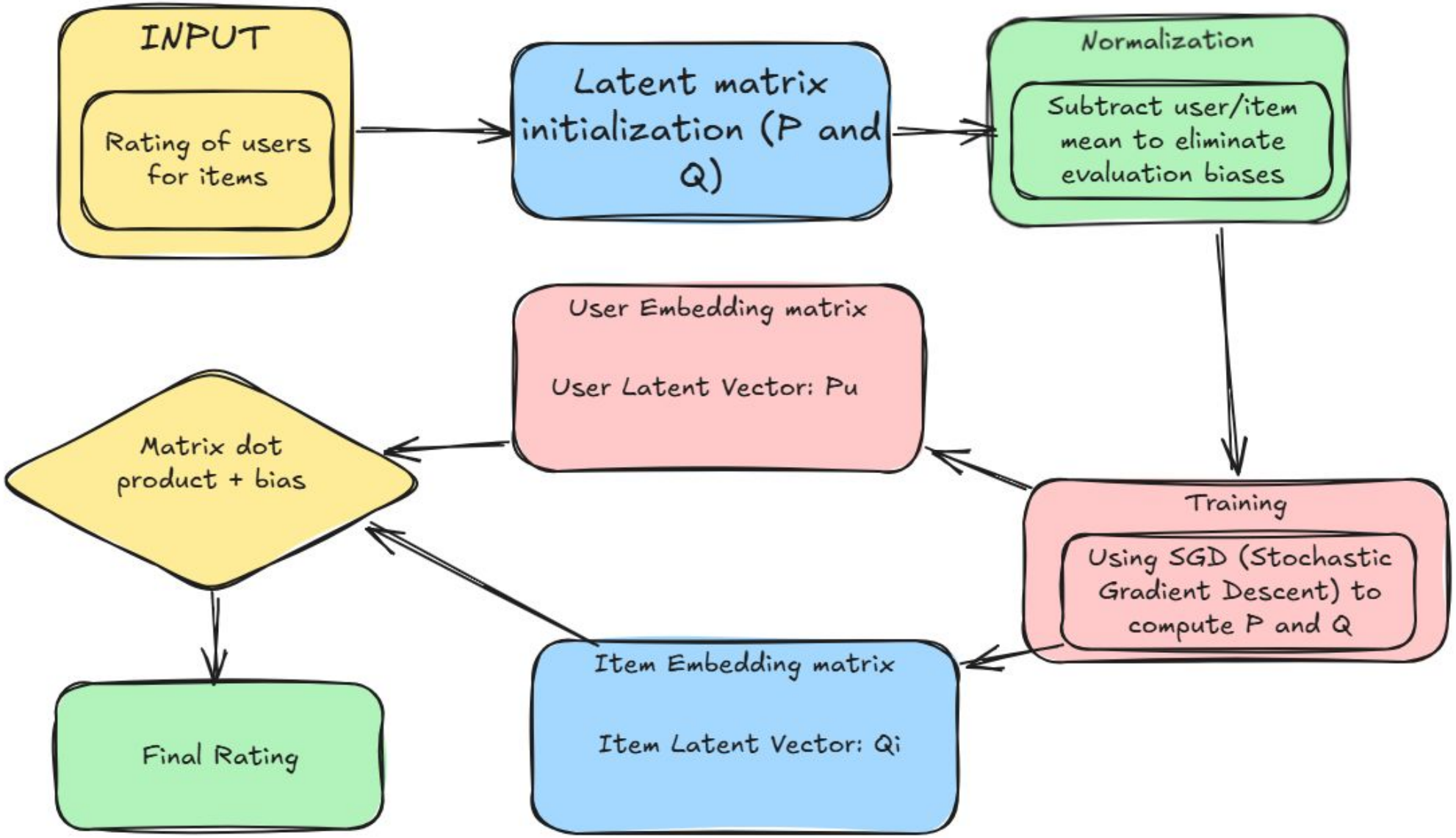
Input: Rating from each users for each movies (Rating matrix)

Intuition: Items are represented by latent feature vectors **q**, while users are represented by latent preference vectors **p**, describing their hidden characteristics and preferences, and the Utility matrix can be computed as follow:

$$\mathbf{R} \approx \hat{\mathbf{R}} = \mathbf{PQ}$$



Architecture

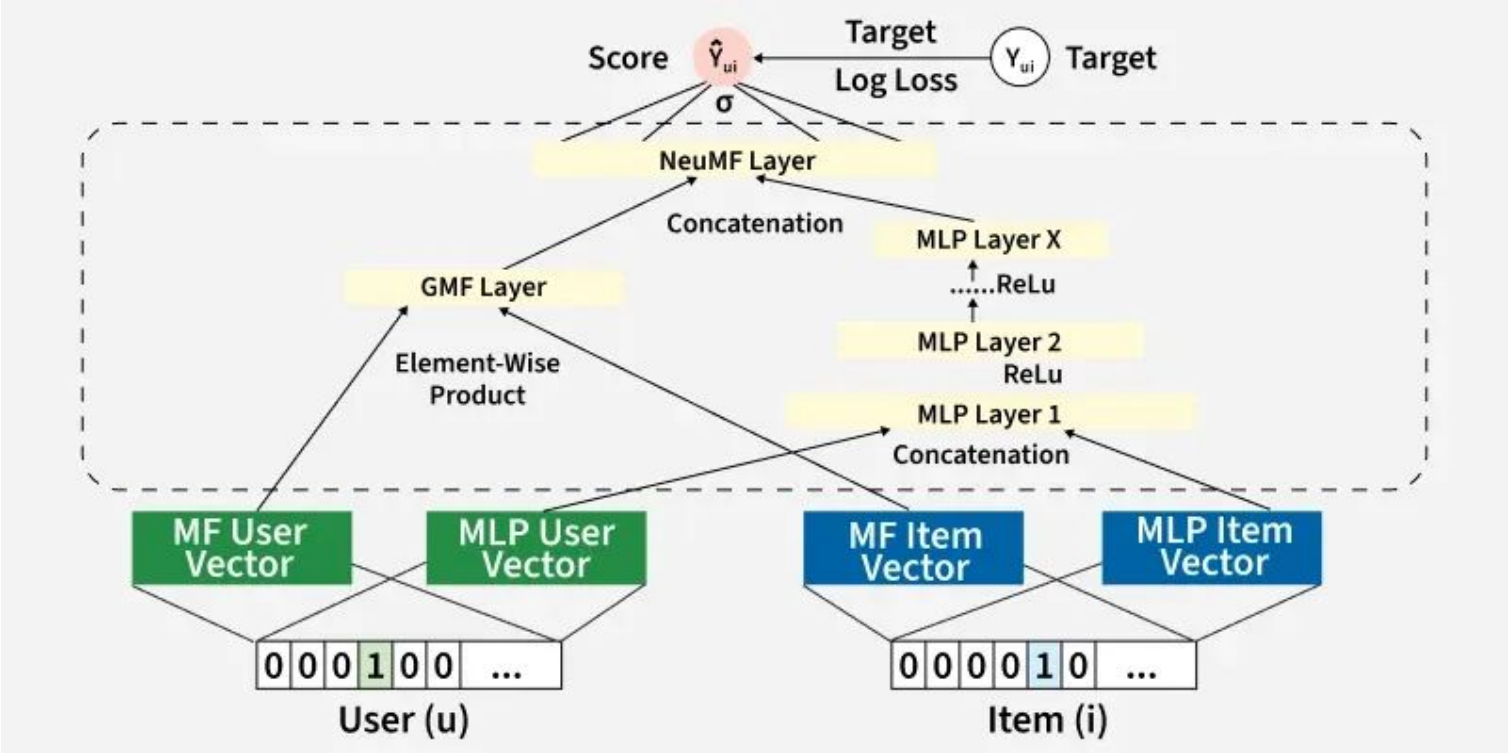


Neural Collaborative Filtering

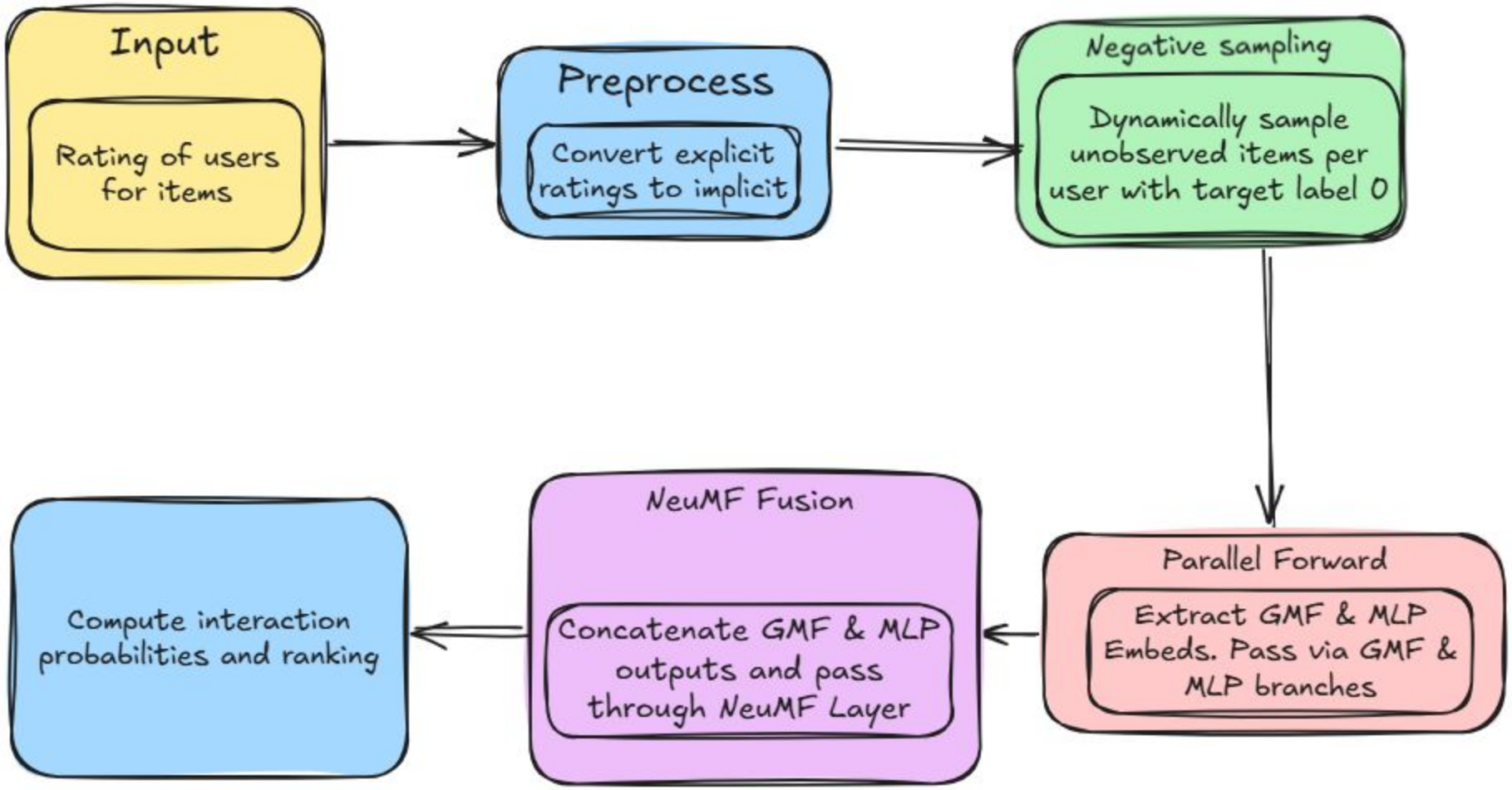
Overview

Idea: User-item interactions are strictly non-linear or linear -> integrating both perspectives

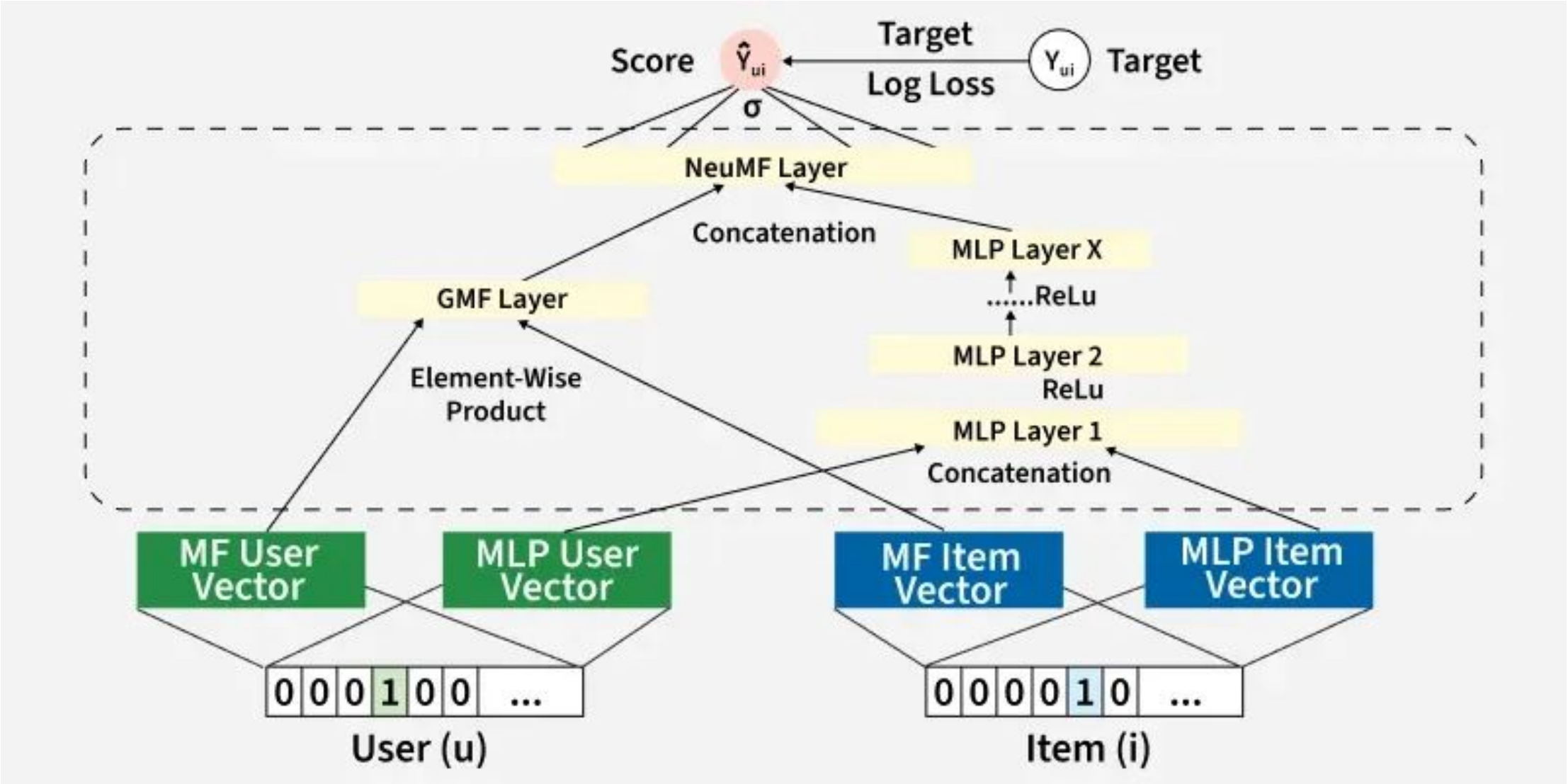
Input: Does the user show positive engagement with the item?



Architecture



Architecture



Two-tower

Two-tower Recommender System

- Implement a **new architecture:** Two-tower Recommender System
- **Idea**: user features and items features are embedded to the same dimension
- **Dot product** to find similarity
- **Updates the parameters** based on the loss function

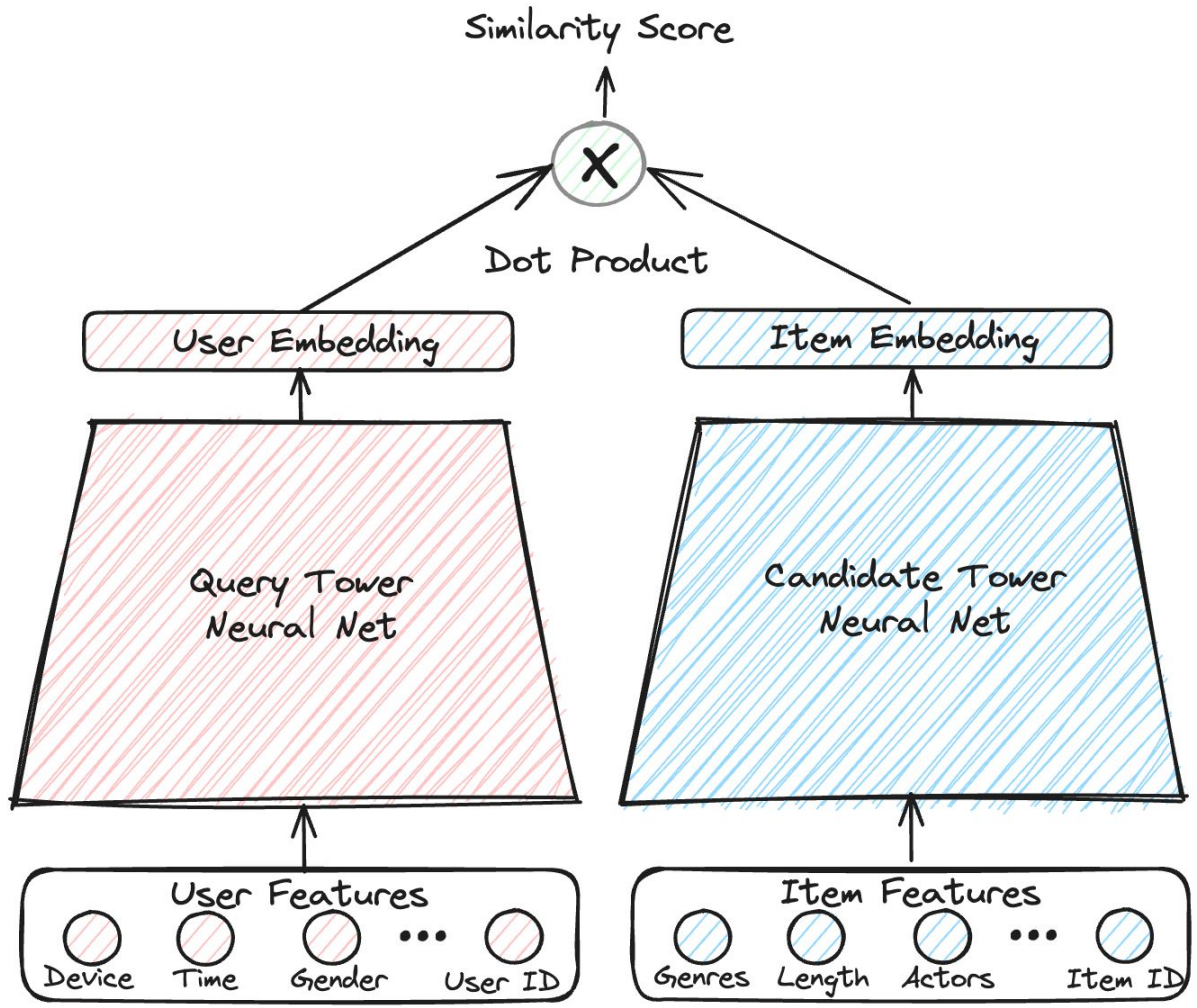


Figure 2: *Two-tower Recommender System*

Complete pipeline

- The complete pipeline of our architecture can be expressed by the following diagram:

Raw data is filtered and history for each user is created

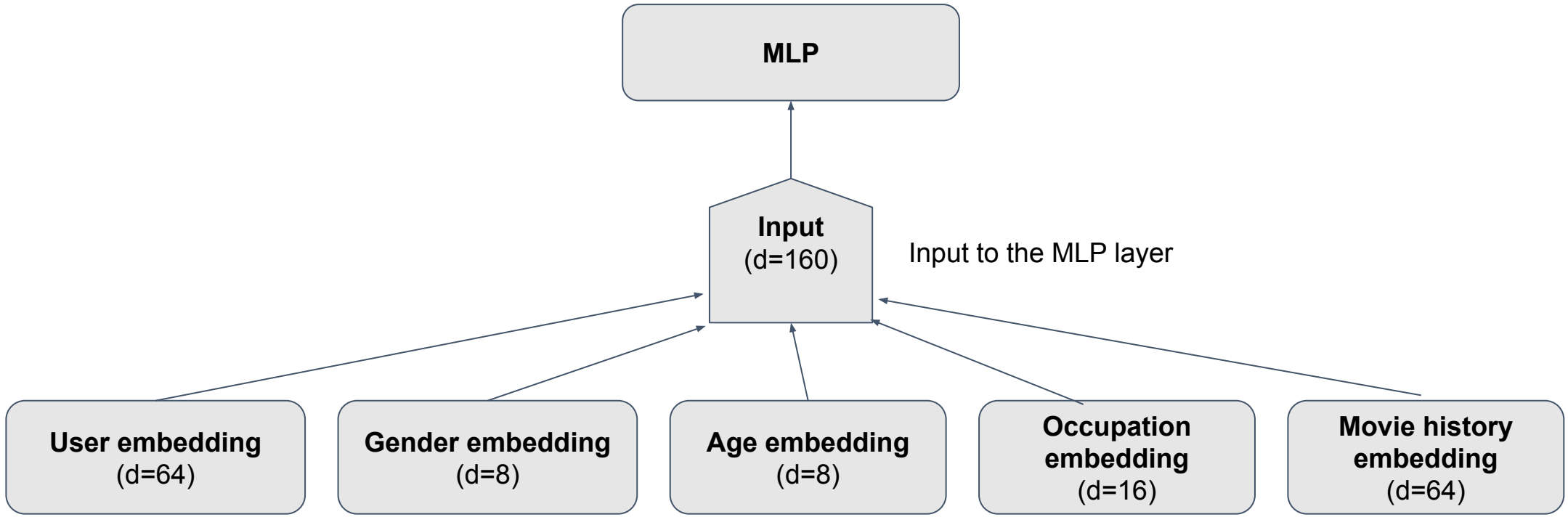
Preprocessed data is fetched into the Two-tower system and outputs the 2 vectors with the same dimensionality

Dot product between 2 vectors is calculated as a similarity metric

Use in-batch softmax loss function

Module Two-tower: User tower

- **Learning representation:** users
- **User features** (id, age, occupation, …) is fetched to the user tower
- Applied a **positional encoding** to movie history (seq_len = 20, each has d=64)



User tower: Transformer Encoder

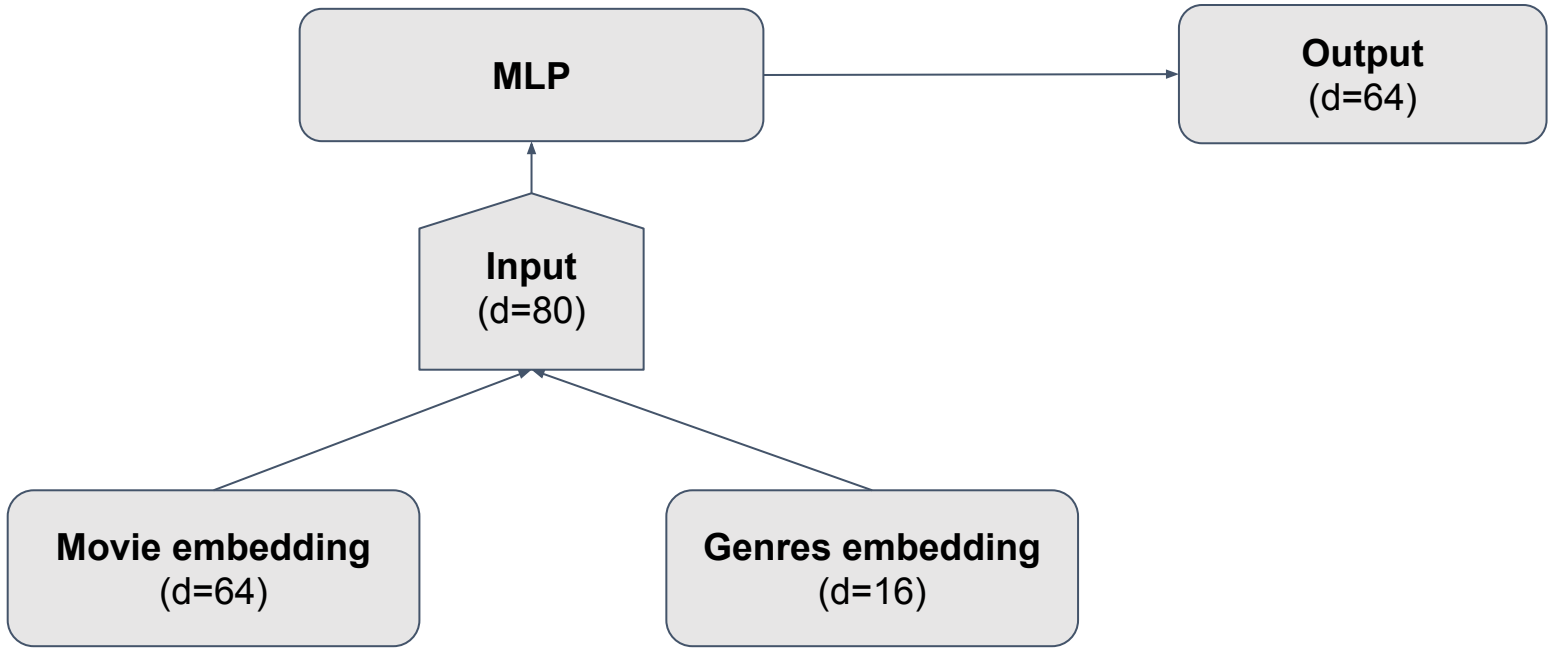
- By default, user_history has shape **(20, 64)**
- Using **multi-head self-attention** with 4 heads, 1 layer
- **Learning representation**: learn the correlation between features in time
- The output is of shape **(20,64). Masked mean pooling** applied → d=64

```
encoder_layer = nn.TransformerEncoderLayer(
    d_model=64, nhead=4, dim_feedforward=256, dropout=0.2, batch_first=True
)
self.transformer = nn.TransformerEncoder(encoder_layer, num_layers=1)
```

Figure 4: *Encoder block in the User tower*

Module Two-tower: Item tower

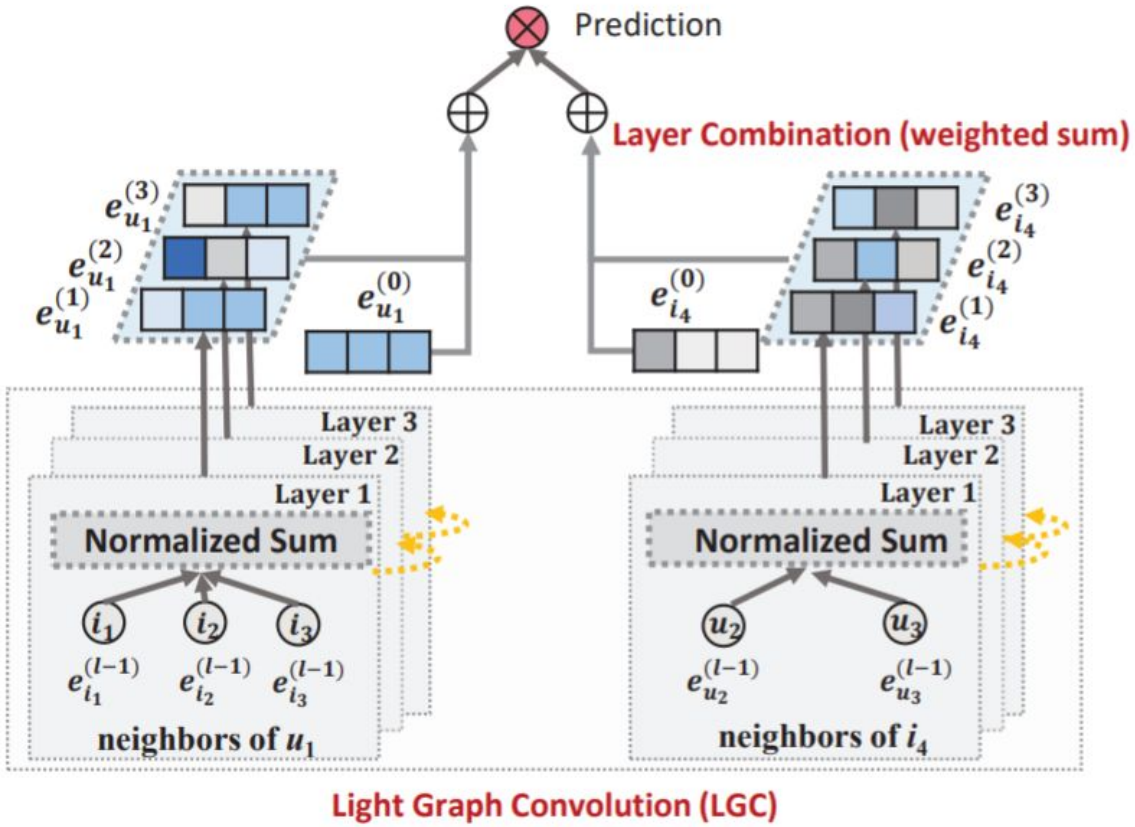
- **Learning representation:** items
- **Item features** (id, genres) is fetched to the item tower. **Mean pooling** of genre embedding for user_history to find the genres embedding for that user



LightGCN

Overview

- Represents user-item interactions as a bipartite graph
- Leverages Graph Convolutional Networks (GCNs) for collaborative filtering



Architecture

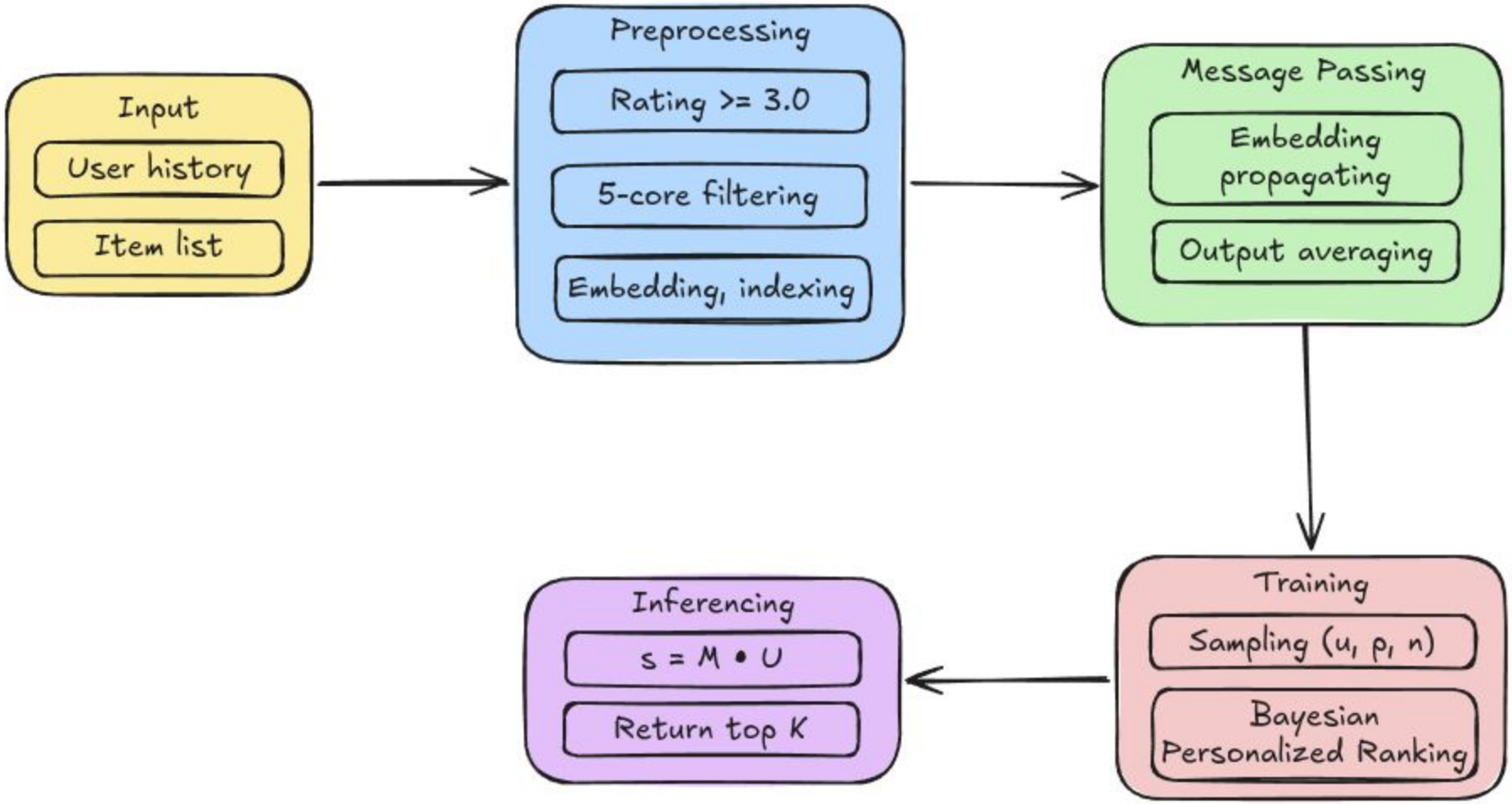
- **Light Graph Convolution:** At layer k, a user's embedding is updated as weighted sum of their connected items' embedding from layer k - 1

$$\mathbf{e}_u^{(k)} = \sum_{i \in \mathcal{N}_u} \frac{1}{\sqrt{|\mathcal{N}_u||\mathcal{N}_i|}} \mathbf{e}_i^{(k-1)}$$

- **Layer Aggregation:** Embeddings at different layers are averaged to form the final embedding:

$$\mathbf{e}_u = \frac{1}{K+1} \sum_{k=0}^{K} \mathbf{e}_u^{(k)}$$

Pipeline


Input
User history
Item list
Preprocessing
Rating >= 3.0
5-core filtering
Embedding, indexing
Message Passing
Embedding propagating
Output averaging
Training
Sampling (u, p, n)
Bayesian Personalized Ranking
Inferencing
s = M • U
Return top K

Self-Attentive Sequential Recommendation

Overview

- **Model architecture: *SASRec (Self-Attentive Sequential Recommendation)***
- **Idea**: User interaction history is modeled as a chronological sequence
- Leverages a **transformer model**: **predict the next item(s)** from a sequence of interacted items for each user

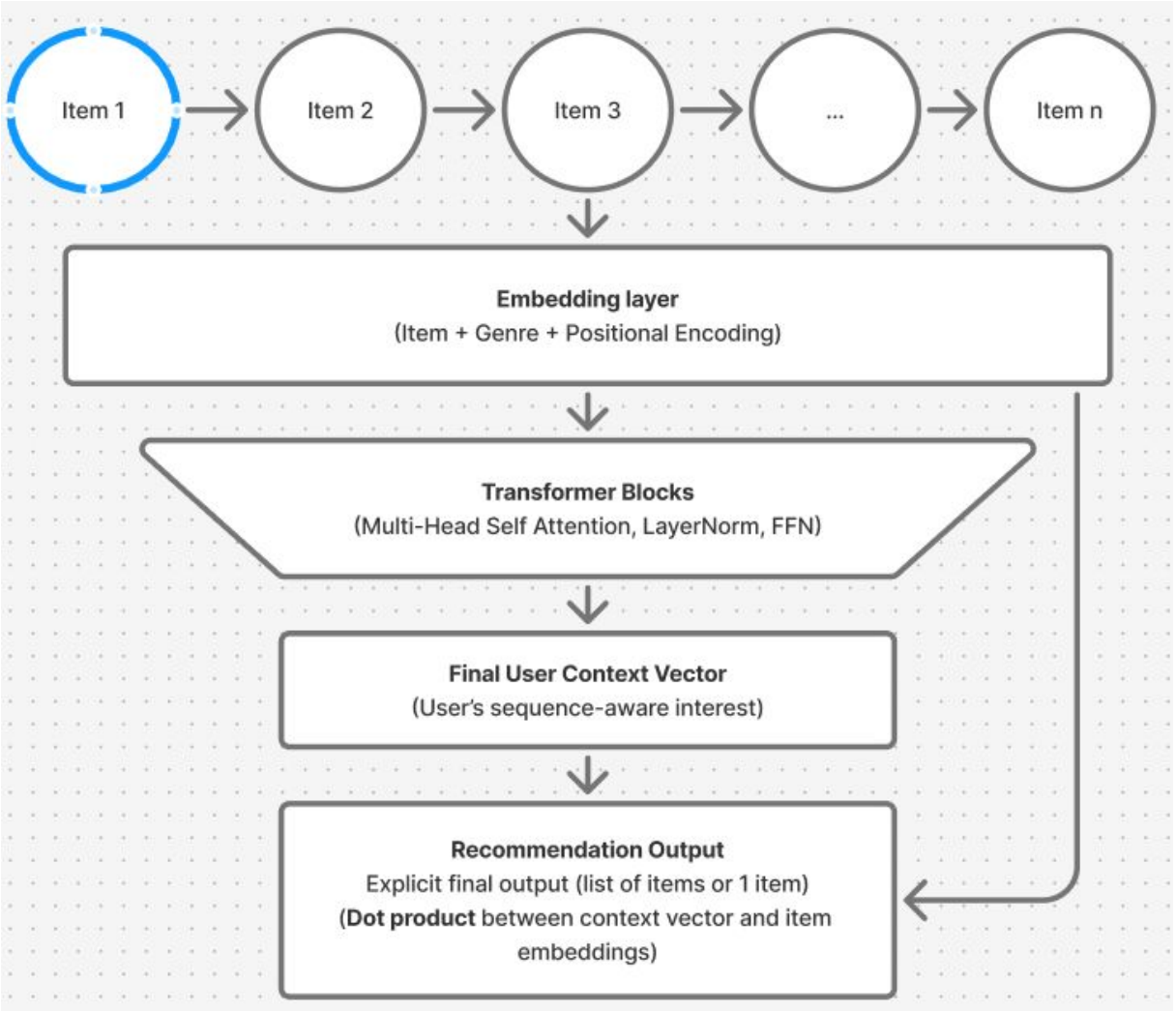


Figure 3: *Overview of SASRec Architecture*

Pipeline

- Preprocess
- Evaluation Protocol

$$\hat{\mathbf{y}}_\ell = \mathbf{h}_\ell^\top \mathbf{E}_{1:N_i}^\top \in \mathbb{R}^{N_i}$$

- Loss function

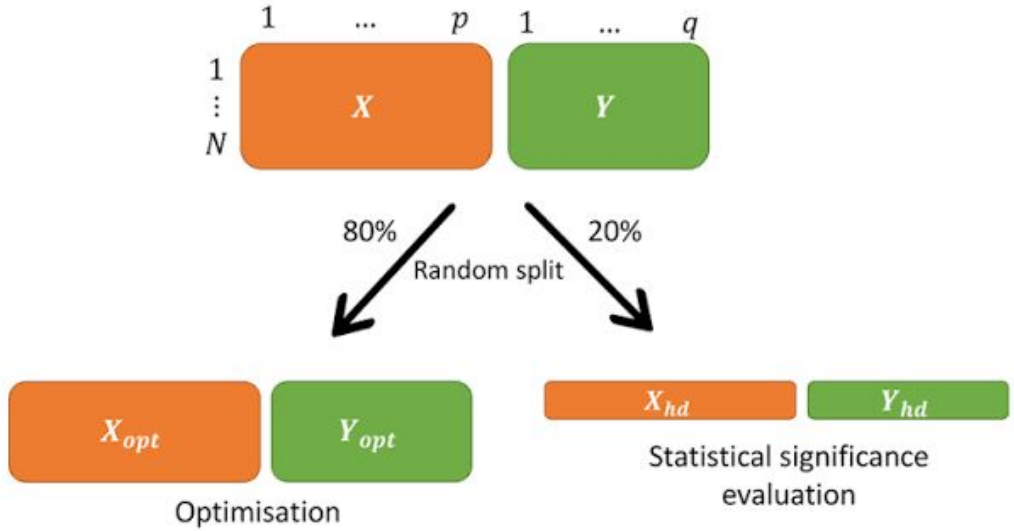
$$\mathcal{L}_u = -\sum_{\ell \in \mathcal{V}_u} \log \frac{\exp(\hat{y}_{\ell i_{\ell+1}})}{\sum_{j=1}^{N_i} \exp(\hat{y}_{\ell j})},$$

Experiments

Setup

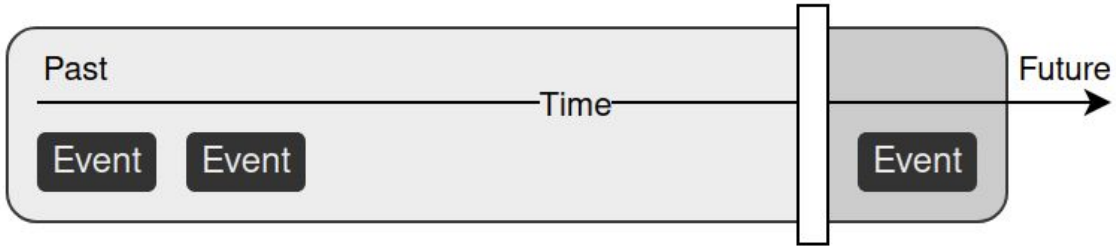
Two data-splitting strategies:

- **Random 80-20 split:** Randomly partition user-item interactions into training-testing set
- **Temporal leave-one-out split:** Most recent interaction of each user is held out for testing



Metrics:

- Top-K: NDCG, MRR, Recall, Precision, HitRate
- Random Split (K = 20), Temporal Split (K = 10)



Results

- Random 80-20:

Model	NDCG@20	Recall@20	MRR@20	Precision@20
Content-based Filtering	0.0185 ± 0.0006	0.0115 ± 0.0006	0.0541 ± 0.0016	0.0152 ± 0.0004
Hybrid Filtering	0.0295 ± 0.0019	0.0328 ± 0.0024	0.0660 ± 0.0018	0.0201 ± 0.0016
Memory-based Filtering	0.0363 ± 0.0008	0.0148 ± 0.0007	0.1014 ± 0.0018	0.0274 ± 0.0012
Matrix Factorization	0.1469 ± 0.0010	0.1275 ± 0.0015	0.2889 ± 0.0038	0.1055 ± 0.0010
Neural CF	0.3264 ± 0.0006	0.2268 ± 0.0006	0.5734 ± 0.0006	0.2460 ± 0.0006
Two Towers	0.0180 ± 0.0003	0.0495 ± 0.0008	0.0096 ± 0.0002	0.0025 ± 0.0001
LightGCN	**0.3578 ± 0.0011**	**0.2604 ± 0.0012**	**0.6077 ± 0.0022**	**0.2630 ± 0.0009**

Results

- Temporal leave-one-out:

Model	NDCG@10	HitRate@10	MRR@10
Content-based Filtering	0.0054 ± 0.0003	0.0108 ± 0.0005	0.0037 ± 0.0007
Hybrid Filtering	0.0086 ± 0.0005	0.0192 ± 0.0005	0.0054 ± 0.0003
Memory-based Filtering	0.0020 ± 0.0002	0.0051 ± 0.0002	0.0011 ± 0.0001
Matrix Factorization	0.0109 ± 0.0004	0.0228 ± 0.0021	0.0073 ± 0.0011
Neural CF	0.0148 ± 0.0004	0.0329 ± 0.0010	0.0094 ± 0.0003
Two Towers	0.0980 ± 0.0024	0.1888 ± 0.0026	0.0706 ± 0.0025
LightGCN	0.0438 ± 0.0003	0.0899 ± 0.0013	0.0302 ± 0.0004
SASRec	**0.1933 ± 0.0027**	**0.3278 ± 0.0048**	**0.1523 ± 0.0023**

Discussion

Performance Disparities

- **Architectural Inductive Biases:** There is no ”one-size-fits-all” architecture

- **Global vs. Sequential:**
  - LightGCN excels at capturing global similarity graphs
  - SASRec thrives on localized, chronological user intent

- **Evaluation Matters:** The validation protocol heavily dictates perceived performance

HUST

hust.edu.vn fb.com/dhbkhn

THANK YOU !